M1 - Programmation de spécialité (UE5.3)

– Projet –

Moteur de Recherche

Analyse, conception, architecture découplée
et étapes de réalisation du projet

Auteur : Evan DYNAK

Année : 2026–2027

Lien du dépôt : https://github.com/Ashzuu/SearchEngine

Résumé

Ce rapport rend compte de l'analyse, la conception et la réalisation d'un moteur de recherche, développé dans le cadre de l'UE5.3 de Master 1. Le système extrait des publications textuelles depuis deux sources de données : Reddit et Arxiv, assurant la persistance des données dans un format tabulaire optimisé (format de fichier csv), puis applique une indexation vectorielle basée sur le schéma de pondération TF-IDF et le stockage en matrices. L'interrogation par requête utilisateur repose sur la similarité cosinus. L'application est structurée selon une architecture client-serveur moderne découplée : un back-end en Python (FastAPI) exposant une API REST stricte et un front-end en Angular (version 22) conçu en *Single Page Application*. Ce document retrace en détail la démarche d'architecture, les pistes de réflexion, les algorithmes et les tests.

Table des matières

1 Les spécifications **4**
1.1 Contexte et problématique . . . 4
1.2 Accès au projet . . . 4
1.3 Spécifications fonctionnelles . . . 4
1.4 Spécifications non-fonctionnelles . . . 5

2 L'analyse **6**
2.1 Environnement de travail retenu . . . 6
2.1.1 Back-end : Python 3 et FastAPI . . . 6
2.1.2 Front-end : Angular . . . 6
2.2 Architecture de classes . . . 7
2.3 Organisation détaillée du front-end . . . 7
2.4 Justification des choix technologiques et architecturaux . . . 8
2.4.1 Pourquoi le patron Singleton ? . . . 8
2.4.2 Pourquoi le patron Factory Method (`DocumentFactory`) ? . . . 8
2.4.3 Patron Repository et interface `DataConnectorInterface` ? . . . 8
2.4.4 Pourquoi les Data Transfer Objects (DTO) ? . . . 8
2.5 Algorithmes spécifiques à l'application . . . 9
2.5.1 Nettoyage et normalisation textuelle . . . 9
2.5.2 Construction du vocabulaire et calcul des fréquences . . . 9
2.5.3 Schéma de pondération TF-IDF . . . 9
2.6 Problèmes rencontrés et solutions apportées . . . 10
2.7 Exemple commenté d'utilisation du programme . . . 11

3 Étapes de réflexion et de réalisation **13**
3.1 Fondations orientées objet et collecte multi-sources (Branche part-1) . . . 13
3.1.1 10 et 15 septembre 2026 . . . 13
3.1.2 17 septembre 2026 . . . 13
3.1.3 07 octobre 2026 . . . 13
3.1.4 08 octobre 2026 . . . 13
3.2 Mathématisation vectorielle et moteur de recherche (Branche part-2) . . . 14
3.2.1 02 novembre 2026 . . . 14
3.3 Interface moderne Angular et refactorisation mono-repo . . . 15
3.3.1 06 novembre 2026 . . . 15

4 La validation **16**
4.1 Procédures de tests unitaires . . . 16

4.1.1 Tests unitaires du back-end Python 16
4.2 Tests globaux via l'interface utilisateur 17

5 La maintenance et les perspectives d'évolution 18
5.1 Évolutions fonctionnelles envisageables 18
5.1.1 Intégration de nouvelles sources documentaires (ex. HAL, Wikipedia) 18
5.1.2 Filtrage multicritère (par date, source ou auteur) 18
5.1.3 Pagination des résultats 18
5.2 Amélioration de l'IHM 18
5.3 Évolutions techniques envisageables 19
5.3.1 Ajout d'autres thèmes de recherche 19
5.3.2 Remplacement de la méthode de stockage de données 19

6 Utilisation de l'IA 20

7 Conclusion 21

1 Les spécifications

1.1 Contexte et problématique

L'objectif est de comprendre l'ensemble des processus permettant qu'un moteur de recherche fonctionne. De ce fait, il a été décidé de réaliser un moteur de recherche simpliste sur un domaine de recherche particulier : *Machine Learning*, avec une interface web très légère, dans le but de manipuler les notions mathématiques et la gestion des données. Selon ce contexte, les étapes de réalisation ont été les suivantes :

- Récolter des corpus documentaires réels issus du Web à partir d'APIs publiques ;
- Modéliser et structurer ces données selon les principes de la programmation orientée objet ;
- Représenter mathématiquement le corpus sous forme vectorielle afin de permettre une recherche par pertinence plutôt qu'une simple recherche exacte de motifs ;
- Fournir une interface de consultation intuitive pour les utilisateurs finaux.

1.2 Accès au projet

Le projet est sur Github, sur le même lien que donné précédemment.

En cas de problématique lié à l'accès du projet, vous pouvez me contacter via le lien suivant : Envoyer un mail (Mail étudiant)

1.3 Spécifications fonctionnelles

Le cahier des charges du logiciel définit sept exigences fonctionnelles majeures :

SF1. Collecte multi-sources : Extraire automatiquement des documents à partir d'au moins deux plateformes externes distinctes avec des volumétries représentatives. Pour ce projet, les deux sources ciblées sur la thématique du *Machine Learning* sont :

- L'API Reddit, via le wrapper officiel `praw`, interrogeant le subreddit spécialisé `r/MachineLearning` ;
- L'API publique d'ArXiv, via le protocole REST/HTTP et l'analyse syntaxique de flux XML (`xmltodict`).

SF2. Persistance et mise en cache des données : Consigner les données collectées dans un fichier structuré (`data.csv`) avec délimiteur tabulation afin d'assurer l'indépendance vis-à-vis du réseau lors des lancements ultérieurs.

SF3. Prétraitement et nettoyage : Normaliser la casse, éliminer les caractères parasites, les tabulations, les sauts de ligne et la ponctuation superflue afin d'homogénéiser le texte.

SF4. Construction de l'index vectoriel : Établir le vocabulaire global du corpus, calculer les fréquences de termes (*Term Frequency*, TF) et les fréquences de documents (*Document Frequency*, DF), puis synthétiser une matrice TF-IDF.

SF5. Moteur de recherche par similarité cosinus : Recevoir une requête utilisateur textuelle, la projeter dans l'espace vectoriel du vocabulaire, calculer la similarité cosinus avec l'ensemble des documents et renvoyer la liste triée des n meilleurs résultats.

SF6. Concordancier et statistiques lexicales : Offrir la possibilité d'analyser le contexte immédiat (gauche et droit) d'un motif textuel donné, ainsi que d'extraire la distribution statistique des mots les plus fréquents.

SF7. Interface graphique interactive : Proposer une application Web permettant à un utilisateur lambda de saisir sa requête et de visualiser instantanément les métadonnées et extraits des documents trouvés.

1.4 Spécifications non-fonctionnelles

SNF1. Efficacité spatiale et temporelle : L'indexation vectorielle sur un vocabulaire comprenant des dizaines de milliers de termes ne doit pas saturer la mémoire vive. Le recours à des matrices creuses s'impose.

SNF2. Modularité et patrons de conception : Le code doit respecter les principes SOLID, en séparant la couche d'accès aux données (*Repository*), la logique métier (*Services*), la représentation (*Models* et *DTOs*) et l'exposition d'API (*Controllers*).

SNF3. Architecture découplée : Le back-end et le front-end doivent fonctionner en processus indépendants, échangeant exclusivement par HTTP/JSON au moyen d'un contrat d'interface stable.

SNF4. Unicité au démarrage : Le processus de récupération des données doit vérifier l'état du stockage local afin d'éviter les surcoûts d'appels réseau ou le duplicata des données.

2 L'analyse

2.1 Environnement de travail retenu

2.1.1 Back-end : Python 3 et FastAPI

Pour la réalisation du back-end, le langage **Python 3.14** associé au framework web **FastAPI** :

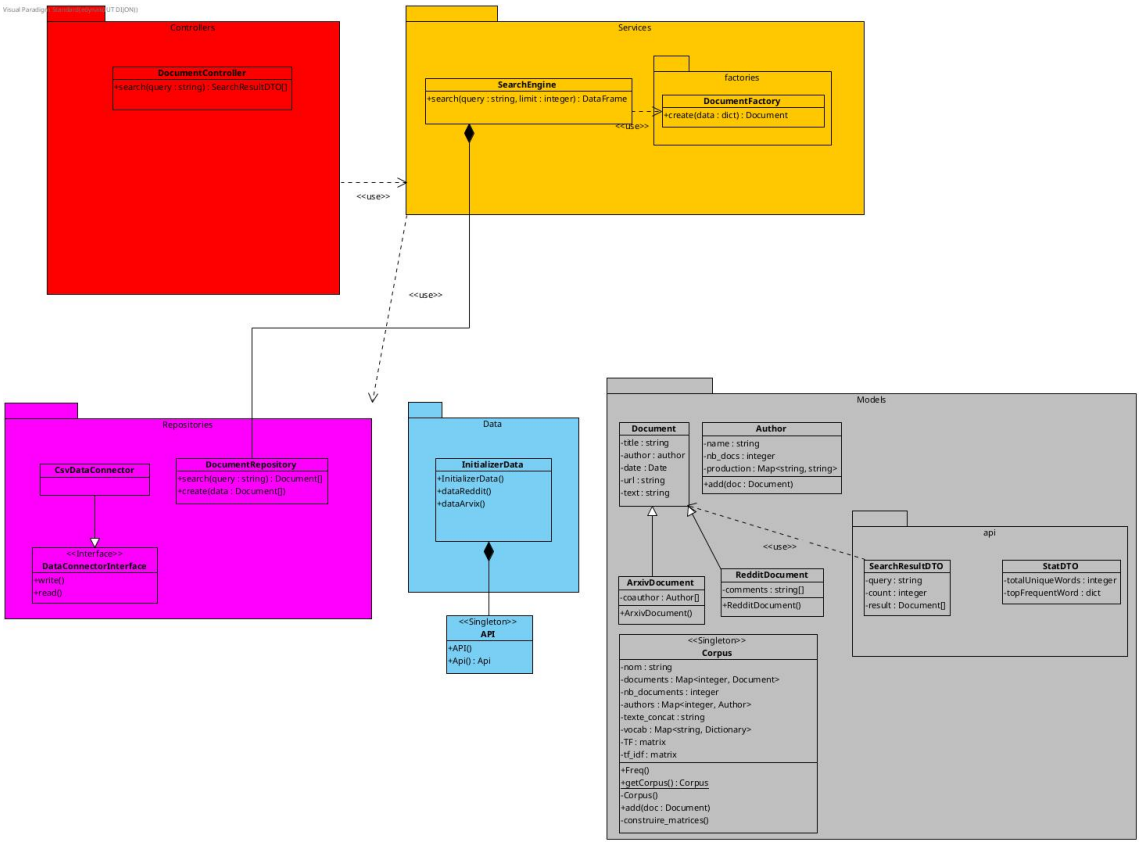
- **Langage imposée par le sujet**
- **Utilisation de FastAPI** : Le choix de cette bibliothèque est liée au fait qu'elle apporte tous les éléments nécessaires pour la création d'une API simple et rapide, permettant de ne pas avoir à recréer ceux-ci.

2.1.2 Front-end : Angular

Pour la réalisation du front-end, le langage **TypeScript** associé au framework web **Angular v22**

- **Modularité des composants et des fonctionnalités** : Angular propose une modulation très forte des fonctionnalités et dépendances grâce aux injections de dépendances intégrées au framework.
- **Facilité de maintenance** : La simplicité de la maintenance d'Angular permet de limiter la dette technique des applications développées.
- **Rapidité** : Angular réalise des applications qui peuvent être soit côté client, soit côté serveur : dans ce projet, il a été choisi d'utiliser le mode "côté client" car cela permet d'avoir des meilleures performances : en effet, tous les calculs sont faits via le serveur Python, de ce fait il n'est pas nécessaire d'avoir également des calculs pour le front côté serveur.

2.2 Architecture de classes



2.3 Organisation détaillée du front-end

L'application front-end réside dans le répertoire `front/` et exploite les dernières avancées d'Angular (version 22) :

- **Composants autonomes** : Suppression des modules complexes (`NgModule`) au profit de composants légers et indépendants.
- **Gestion d'état réactive par Signals** : Utilisation des primitives `signal`, `WritableSignal` et `computed` pour réagir aux flux de données sans surcharge de cycles de détection.
- **Intercepteur HTTP et indicateur de chargement** : L'intercepteur fonctionnel `httpInterceptorInterceptor` intercepte chaque requête sortante, incrémente le compteur du `LoaderService`, et le décrémente lors de la finalisation, permettant de connaitre l'état de chargement de l'application.
- **Composant partagé `SearchBar`** : Zone de saisie avec liaison bidirectionnelle (`[(ngModel)]`), capture des événements clavier (ENTRÉE).

2.4 Justification des choix technologiques et architecturaux

2.4.1 Pourquoi le patron Singleton ?

La construction de l'index pour plusieurs milliers de documents représente un coût mémoire et processeur non négligeable. Si chaque requête HTTP devait réinstancier le `Corpus` ou recharger le fichier CSV, l'API s'effondrerait sous la charge. L'application du patron **Singleton** à la classe `Corpus` et à la classe `Api` garantit l'unicité de l'état en mémoire et partage la même structure d'indexation entre tous les appels concurrents.

2.4.2 Pourquoi le patron Factory Method (`DocumentFactory`) ?

Les données brutes lues depuis le fichier CSV contiennent un champ permettant de les différencier : `origin` (`"reddit"` ou `"arxiv"`). Confier l'instanciation directe au code consommateur créerait un couplage fort et multiplierait les blocs conditionnels. La `DocumentFactory` centralise la logique de construction et de désérialisation spécifique, garantissant l'extensibilité du code.

2.4.3 Patron Repository et interface `DataConnectorInterface` ?

L'accès au stockage local est isolé derrière l'interface abstraite `DataConnectorInterface`. Actuellement implémentée par `CsvDataConnector`, cette architecture permettrait de basculer sans aucun impact sur le reste de l'application vers un stockage SQLite, PostgreSQL ou MongoDB simplement en fournissant une nouvelle classe concrète.

2.4.4 Pourquoi les Data Transfer Objects (DTO) ?

Initialement, le moteur de recherche renvoyait directement des structures Pandas ou des dictionnaires non typés. L'introduction de `SearchResultDTO` et `StatDTO` a permis de stabiliser le contrat d'interface exposé au client Angular, protégeant le front-end contre toute modification de l'implémentation interne du back-end. En cas d'ajout de nouvelles fonctionnalités, il sera certes nécessaire de devoir modifier les objets DTOs, toutefois, ils permettent d'être certain d'obtenir les données attendues, ni plus ni moins.

2.5 Algorithmes spécifiques à l'application

2.5.1 Nettoyage et normalisation textuelle

L'algorithme de prétraitement (`TextCleaner`) conditionne la qualité de l'indexation. Il applique les transformations successives :

1. Passage intégral en minuscules pour éliminer la sensibilité à la casse.
2. Remplacement systématique des sauts de ligne (`\r\n`, `\n`) et tabulations par des espaces simples.
3. Application d'une expression régulière de filtrage `[^\w\s]|[\d_]` pour éliminer ponctuation, symboles spéciaux et chiffres.
4. Réduction des espaces blancs contigus à une seule espace et suppression des espaces terminales (*stripping*).

2.5.2 Construction du vocabulaire et calcul des fréquences

La méthode `_calculer_frequences_et_vocabulaire()` parcourt chaque document du corpus :

- Pour chaque mot t d'un document d, la fréquence de terme globale $\mathrm{TF}(t)$ est incrémentée.
- Un ensemble de mots uniques par document est conservé afin d'incrémenter la fréquence de document $\mathrm{DF}(t)$ au plus une fois par document.
- Les termes sont triés par ordre alphabétique pour attribuer un identifiant unique de colonne.

2.5.3 Schéma de pondération TF-IDF

Pour chaque terme t associé à sa colonne j, le poids IDF est calculé selon la formulation suivante :

$$\mathrm{IDF}(t) = \ln\left(\frac{1+N}{1+\mathrm{DF}(t)}\right) + 1 \tag{1}$$

où N désigne l'effectif global des documents dans le corpus. La matrice globale $\mathbf{M}_{\text{TF-IDF}}$ est ensuite calculée par multiplication colonne à colonne :

$$\mathbf{M}_{\text{TF-IDF}} = \mathbf{TF} \odot \vec{\mathrm{IDF}} \tag{2}$$

2.6 Problèmes rencontrés et solutions apportées

P1. Instanciation répétée et bug de décorateur Singleton :
Problème : La première implémentation du décorateur singleton stockait l'instance dans une liste globale mal réinitialisée, provoquant des erreurs lors des tests ou des exécutions imbriquées.
Solution : Refactorisation complète du décorateur avec un dictionnaire interne `instances` indexé par classe, garantissant une conservation sûre de l'instance unique.

P2. Temps de recalcul de l'index vectoriel à l'ajout de documents :
Problème : Reconstruire le vocabulaire et recalculer les matrices creuses à chaque ajout de document individuel pénalisait les temps de réponse de manière inacceptable.
Solution : Implémentation du patron de chargement paresseux (*Lazy Loading*) via des `@property` avec mise en cache. Les calculs ne sont déclenchés que lors du premier accès en lecture. L'ajout d'un document réinitialise simplement les caches à `None`.

P3. Épuisement des quotas d'appels API et dépendance réseau :
Problème : Interroger les API Reddit et arXiv à chaque lancement de l'application engendrait une forte latence et risquait d'induire un blocage IP pour requêtes excessives.
Solution : La classe `InitializerData` vérifie au préalable si le fichier `data.csv` existe et contient des données exploitables. La collecte distante n'est exécutée qu'en cas d'absence complète de données locales.

P4. Blocage CORS entre le client Angular et le serveur FastAPI :
Problème : En phase d'intégration du front-end, le navigateur bloquait les requêtes émises depuis l'origine `http://localhost:4200` vers l'API sur `http://localhost:8000`.
Solution : Intégration du middleware `CORSMiddleware` dans FastAPI, autorisant explicitement les requêtes en provenance de `http://localhost:4200` avec les méthodes et en-têtes nécessaires.

P5. Organisation confuse lors du passage au Front-end :
Problème : La racine du projet mélangeait scripts Python, fichiers de dépendances et configuration globale, ce qui rendait impossible l'installation propre d'une application Angular.
Solution : Restructuration complète du dépôt sous forme de mono-repo, délimitant strictement les dossiers `back/` (Python, FastAPI, dépendances pip) et `front/` (TypeScript, Angular, node_modules).

2.7 Exemple commenté d'utilisation du programme

Voici le scénario complet d'une session de recherche standard :

1. **Amorçage du serveur d'application** :

   ```
   $ cd back/
   $ fastapi dev main.py
   ```

   Le composant `Api` est instancié. `InitializerData` valide l'intégrité de `data.csv`. `CsvDataConnector` lit les enregistrements et `DocumentFactory` peuple le singleton `Corpus`. L'API démarre sur `http://127.0.0.1:8000` (ou `http://localhost:8000`).

2. **Lancement de l'interface utilisateur** :

   ```
   $ cd front/
   $ npm start
   ```

   Le serveur de développement Angular compile l'application et sert l'interface sur `http://localhost:4200`.

3. **Navigation et saisie de la requête** : L'utilisateur se rend sur `http://127.0.0.1:4200` (ou `http://localhost:4200`). La page d'accueil affiche la barre de recherche. L'utilisateur renseigne les termes « `reinforcement learning robotics` » et valide avec la touche Entrée.

4. **Traitement et transition dans l'application** : Le composant `SearchBar` déclenche la navigation vers `/search?query=reinforcement+learning+robotics`. La page `SearchingPage` est activée. Elle sollicite `SearchService.search()`, qui émet la requête HTTP correspondante. L'intercepteur HTTP active le signal d'état de chargement dans `LoaderService`.

5. **Résolution sur le serveur** : Le contrôleur `SearchEngineController` reçoit la requête, la transmet au `SearchEngine`. Ce dernier nettoie la chaîne, calcule le produit scalaire sur la matrice creuse TF-IDF et classe les meilleurs documents. Le contrôleur renvoie un objet `SearchResultDTO` sérialisé en JSON :


```
{
  "query": "reinforcement learning robotics",
  "count": 10,
  "result": [
    {
      "id": 112,
      "score": 0.7421,
      "titre": "Deep Reinforcement Learning for Robot Navigation",
      "origine": "arxiv",
      "url": "http://arxiv.org/abs/2103.xxxxx",
      "auteur": "Smith et al.",
      "texte": "This paper presents a novel model-free RL algorithm..."
    }
  ]
}
```


6. **Affichage et exploitation du résultat** : Le flux HTTP se clôture, le `LoaderService` rétablit l'affichage normal. La directive structurelle `@for` restitue les cartes des dix documents les plus pertinents, avec leur score, leur titre cliquable ouvrant la source originale (arXiv ou Reddit) et un extrait représentatif de leur contenu.

3 Étapes de réflexion et de réalisation

3.1 Fondations orientées objet et collecte multi-sources (Branche part-1)

3.1.1 10 et 15 septembre 2026

Poser les bases d'un environnement de travail propre. Mise en place du fichier `.gitignore` pour exclure les fichiers d'environnement Python ou de cache.

3.1.2 17 septembre 2026

- Une classe abstraite de base `Document` regroupant les propriétés : titre, auteur, date, url, texte.
- Des sous-classes spécialisées (`RedditDocument`, `ArvixDocument`) enrichies d'autres attributs ajoutés dans leur source d'origine.
- La classe `Corpus` agissant comme conteneur des documents.
- Une première abstraction de persistance avec l'interface `DataConnectorInterface` et son implémentation `CsvDataConnector`.
- Une première tentative de mise en œuvre du patron `Singleton`.

3.1.3 07 octobre 2026

- Intégration du framework **FastAPI** et du serveur **Uvicorn**.
- Découplage de la couche d'exposition via une interface `IController` et un contrôleur dédié `SearchEngineController`.
- Création de l'entité `Api` pour encapsuler l'application web.

3.1.4 08 octobre 2026

- Implémentation effective de l'extraction sur ArXiv et sur Reddit.
- Conception d'un mécanisme de cache local vérifiant si `data.csv` est déjà alimenté avant d'effectuer de nouveaux appels réseau.
- Correction du décorateur `Singleton` devenu parfaitement fiable.
- Introduction du patron **Factory Method** (`DocumentFactory`) permettant d'isoler l'analyse du champ `origin` et d'instancier la classe concrète adaptée.

3.2 Mathématisation vectorielle et moteur de recherche (Branche part-2)

3.2.1 02 novembre 2026

- Introduction du module autonome `TextCleaner` avec sa méthode `nettoyer_texte`.
- Implémentation du concordancier par expressions régulières (`Corpus.concorde`).
- Dérivation du vocabulaire global et calcul des fréquences globales (TF) et documentaires (DF).

Implémentation complète de `SearchEngine` :

- Vectorisation à la volée de la requête utilisateur sur la base du vocabulaire indexé et des coefficients IDF.
- Calcul vectorisé de la similarité cosinus par produit matriciel creux.
- Extraction des n meilleurs scores et enrichissement des résultats avec métadonnées complètes.
- Exposition des deux routes REST principales : `/search` et `/stats`.

3.3 Interface moderne Angular et refactorisation mono-repo

3.3.1 06 novembre 2026

- Choix de développer un front-end complet en **Angular 22** (*Single Page Application*).
- Réorganisation du dépôt en **mono-repo** séparé en deux dossiers hermétiques : `back/` et `front/`.
- Introduction des DTOs (`SearchResultDTO`, `StatDTO`) pour séparer le modèle interne de calcul du schéma JSON envoyé au client.
- Sécurisation des flux inter-domaines par l'ajout de `CORSMiddleware` dans FastAPI autorisant `http://localhost:4200`.
- Conception d'une interface responsive avec gestion d'état réactive (Angular Signals) et intercepteur HTTP pour le retour visuel de chargement.

4 La validation

4.1 Procédures de tests unitaires

4.1.1 Tests unitaires du back-end Python

Les méthodes critiques du back-end ont fait l'objet de vérifications unitaires ciblées :

- **Validation du nettoyage textuel (`TextCleaner`)** : Vérification de la suppression des caractères accentués parasites, des tabulations, des retours chariot multiples et de la conservation stricte des mots-clés alphabétiques.
- **Validation de la fabrique (`DocumentFactory`)** : Contrôle de l'instanciation rigoureuse d'un `RedditDocument` lorsque `origin == "reddit"` et d'un `ArvixDocument` lorsque `origin == "arxiv"`, avec décodage correct des co-auteurs en tableau.
- **Validation du calcul matriciel creux** : Vérification que la matrice TF-IDF produite a les bonnes dimensions et que ses éléments sont strictement non négatifs.
- **Validation interactive des endpoints FastAPI** : Utilisation de l'interface Swagger générée sur `http://localhost:8000/docs` pour éprouver les codes d'état HTTP (200, 422 en cas de paramètre manquant) et la conformité des schémas JSON retournés.

4.2 Tests globaux via l'interface utilisateur

TAB. 1 : Cahier de recette et validation des tests globaux de l'interface utilisateur

Scénario de test	Données d'entrée	Comportement attendu et vérifications	Résultat
Recherche nominale simple	`"neural"`	Interception de la saisie, appel à `/search?query=neural`, affichage des 10 documents les plus pertinents avec titres, liens et extraits.	**Conforme**
Recherche composite multi-termes	`"deep learning vision"`	Vectorisation conjointe, classement privilégiant les documents maximisant la similarité cosinus avec l'ensemble des termes.	**Conforme**
Insensibilité à la casse	`"MaChiNe LeArNiNg"`	Normalisation automatique par `TextCleaner` ; résultats et scores strictement identiques à `"machine learning"`.	**Conforme**
Tolérance aux bruits et ponctuation	`"AI & robotics!?#"`	Épuration des caractères non alphanumériques superflus ; recherche traitée comme équivalente à `"ai robotics"`.	**Conforme**
Requête vide ou espaces	`""` ou `"  "`	Blocage de la soumission au niveau du composant `SearchBar` ; aucune navigation superflue ni appel réseau déclenché.	**Conforme**
Termes hors vocabulaire	`"unknown123"`	Similarité cosinus nulle sur tout le corpus ; déclenchement du bloc `@empty` avec message convivial invitant à retenter.	**Conforme**
Indicateur de chargement réactif	Requête réseau	Activation synchrone du `LoaderService` via l'intercepteur HTTP ; masquage temporaire des résultats et affichage du spinner.	**Conforme**
Consultation des sources externes	Clic sur un titre de résultat	Exploitation du champ discriminant `url` pour rediriger l'utilisateur vers la publication d'origine (arXiv ou Reddit).	**Conforme**
Recherche consécutive directe	Nouvelle saisie depuis la page	Mise à jour réactive du paramètre d'URL `?query=...` via le routeur Angular ; rafraîchissement immédiat de la liste de résultats.	**Conforme**

5 La maintenance et les perspectives d'évolution

5.1 Évolutions fonctionnelles envisageables

5.1.1 Intégration de nouvelles sources documentaires (ex. HAL, Wikipedia)

- **Mise en place** : Créer une sous-classe `HalDocument` étendant `Document`, ajouter une méthode de collecte dans `InitializerData`, et déclarer le cas d'instanciation correspondant dans la `DocumentFactory`.
- **Evaluation de faisabilité : Facile**. Grâce au patron Factory et à l'architecture ouverte de la hiérarchie de classes, aucune modification des algorithmes d'indexation ou des contrôleurs n'est requise.

5.1.2 Filtrage multicritère (par date, source ou auteur)

- **Mise en place** : Étendre les paramètres de l'endpoint `/search` dans `SearchEngineController` avec des options optionnelles (`origin`, `author`, `date_from`) et appliquer un masque de filtrage vectoriel sur le tableau de résultats final.
- **Evaluation de faisabilité : Facile à Moyenne**. Les métadonnées sont déjà conservées dans le conteneur `Corpus` et retournées dans les résultats.

5.1.3 Pagination des résultats

- **Mise en place** : Ajouter un paramètre `page` et `page_size` sur l'API, et utiliser l'opérateur de slicing sur le vecteur ordonné des indices retournés par `numpy.argsort`.
- **Evaluation de faisabilité : Très facile**.

5.2 Amélioration de l'IHM

- **Mise en place** : Créer une charte graphique au site, un logo, permettant de rendre cela plus agréable
- **Evaluation de faisabilité : Facile à Difficile**. Tout dépend des modifications qui sont apportées à l'application, jusqu'où on développe la charte graphique, etc...

5.3 Évolutions techniques envisageables

5.3.1 Ajout d'autres thèmes de recherche

- **Mise en place** : Ajouter d'autres mots clés de recherche lors de la récupération des données via les APIs.
- **Evaluation de faisabilité : Haute**. Facilement mis en place pour plusieurs thèmes, toutefois si le nombre de thèmes est trop grand, risque de baisse de performances du moteur lié à la volumétrie de données dans un format de stockage inadapté à celle-ci.

5.3.2 Remplacement de la méthode de stockage de données

- **Mise en place** : Transformer notre base de données actuelle (fichier csv) en une vraie base de données, ayant de meilleurs performances sur la récupération des données, et notamment pour les tris de données
- **Evaluation de faisabilité : Moyenne**. D'un point de vue, la mise en place est plutôt rapide car il suffit uniquement d'adapter notre implémentation de l'interface `DataConnectorInterface` avec une base de données. Toutefois, côté base de données, il faut recréer les schémas correspondants, bien structuré les tables afin que celle-ci aient un temps de réponse correct, et cela peut prendre un certain temps et une certaine complexité.

6 Utilisation de l'IA

Dans ce projet, l'IA a été utilisé pour les éléments suivants :

1. **Aide dans la mise en place architecturale de l'API en Python** : n'ayant jamais utilisé FastAPI comme librairie, mais souhaitant imposé une architecture maitrisée (Controllers -> Service -> Repository), l'IA m'a permis de mieux structurer mon code pour la suite.

2. **Aide à la mise en place du rapport :** Aide pour la récupération des différents tests / commits depuis le dépôt Git, et aide pour l'utilisation de LaTeX. N'ayant jamais utilisé également cette méthode de création de documents, et ayant connaissance qu'il sera utile dans d'autres matières ainsi que dans le monde de la recherche, celui-ci a permis de créer rapidement la structure du document.

3. **Vérification de la qualité du code** : Régulièrement durant le processus, je demandais à ce que l'IA vérifie la qualité afin que celle-ci ne se détériore pas trop.

7 Conclusion

Ce projet a concrétisé la mise en œuvre d'un moteur de recherche textuel complet, rigoureux et performant. En partant d'un cahier des charges exigeant, le travail a couvert l'ensemble du cycle de vie d'une application logicielle moderne :

1. L'**analyse approfondie du problème** et l'identification des entités et abstractions clés ;
2. La **conception orientée objet** mobilisant des patrons éprouvés (Singleton, Factory, Repository, DTO, Lazy Loading) ;
3. L'**ingénierie mathématique et algorithmique**, conjuguant normalisation textuelle, pondération TF-IDF et calcul matriciel creux ultra-performant sous SciPy ;
4. Le **développement d'une interface web moderne et réactive** en Angular 22, découplée du back-end par une API REST conforme aux standards de l'art ;
5. Une **stratégie de validation rigoureuse** unissant tests unitaires et tests fonctionnels d'intégration.

L'application constitue aujourd'hui un socle solide, documenté et immédiatement extensible vers de nouveaux horizons technologiques.